



CODE SECURITY ASSESSMENT

DEVFUN TOKEN

Overview

Project Summary

- Name: Devfun-Token
- Platform: EVM-compatible chains
- Language: Solidity
- Repository:
 - <https://github.com/devfun-org/devfun-token>
- Audit Range: See [Appendix - 1](#)

Project Dashboard

Application Summary

Name	Devfun-Token
Version	v2
Type	Solidity
Dates	Sep 23 2026
Logs	Sep 21 2026, Sep 23 2026

Vulnerability Summary

Total High-Severity issues	0
Total Medium-Severity issues	0
Total Low-Severity issues	0
Total informational issues	3
Total	3

Contact

E-mail: support@salusec.io

Risk Level Description

High Risk	The issue puts a large number of users' sensitive information at risk, or is reasonably likely to lead to catastrophic impact for clients' reputations or serious financial implications for clients and users.
Medium Risk	The issue puts a subset of users' sensitive information at risk, would be detrimental to the client's reputation if exploited, or is reasonably likely to lead to a moderate financial impact.
Low Risk	The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
Informational	The issue does not pose an immediate risk, but is relevant to security best practices or defense in depth.

Content

Introduction	4
1.1 About SALUS	4
1.2 Audit Breakdown	4
1.3 Disclaimer	4
Findings	5
2.1 Summary of Findings	5
2.2 Informational Findings	6
1. Lack of Security Contact Information for Responsible Disclosure	6
2. Mapping Declaration Lacks Named Parameters for Improved Readability	7
3. Missing events for functions that change critical state	8
Appendix	9
Appendix 1 - Files in Scope	9

Introduction

1.1 About SALUS

At Salus Security, we are in the business of trust.

We are dedicated to tackling the toughest security challenges facing the industry today. By building foundational trust in technology and infrastructure through security, we help clients to lead their respective industries and unlock their full Web3 potential.

Our team of security experts employ industry-leading proof-of-concept (PoC) methodology for demonstrating smart contract vulnerabilities, coupled with advanced red teaming capabilities and a stereoscopic vulnerability detection service, to deliver comprehensive security assessments that allow clients to stay ahead of the curve.

In addition to smart contract audits and red teaming, our Rapid Detection Service for smart contracts aims to make security accessible to all. This high calibre, yet cost-efficient, security tool has been designed to support a wide range of business needs including investment due diligence, security and code quality assessments, and code optimisation.

We are reachable on Telegram (<https://t.me/salusec>), Twitter (https://twitter.com/salus_sec), or Email (support@salusec.io).

1.2 Audit Breakdown

The objective was to evaluate the repository for security-related issues, code quality, and adherence to specifications and best practices. Possible issues we looked for included (but are not limited to):

- Risky external calls
- Integer overflow/underflow
- Transaction-ordering dependence
- Timestamp dependence
- Access control
- Call stack limits and mishandled exceptions
- Number rounding errors
- Centralization of power
- Logical oversights and denial of service
- Business logic specification
- Code clones, functionality duplication

1.3 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release and does not give any warranties on finding all possible security issues with the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues.

Findings

2.1 Summary of Findings

ID	Title	Severity	Category	Status
1	Lack of Security Contact Information for Responsible Disclosure	Informational	Configuration	Resolved
2	Mapping Declaration Lacks Named Parameters for Improved Readability	Informational	Code Quality	Resolved
3	Missing events for functions that change critical state	Informational	Logging	Resolved

2.2 Informational Findings

1. Lack of Security Contact Information for Responsible Disclosure

Severity: Informational

Category: Configuration

Target:

- src/credits/ICBurner.sol
- src/credits/ICDepositFactory.sol
- src/credits/ICGoverned.sol
- src/credits/ICInventoryVault.sol
- src/credits/CIssuanceController.sol

Description

These contracts, e.g. ICBurner, ICDepositFactory do not specify a security contact point. Including a designated security contact (such as an email address or ENS name) in the contract's NatSpec header facilitates responsible vulnerability disclosure. This makes it easier for external researchers to quickly reach the appropriate team in the event a vulnerability is identified, helping minimize the time window between discovery and mitigation. The Ethereum community has begun standardizing this practice using the `@custom:security-contact` tag, adopted by tools such as OpenZeppelin Wizard and ethereum-lists.

Recommendation

Consider adding a NatSpec comment at the top of the contract with a `@custom:security-contact` field pointing to the preferred disclosure channel.

Status

This issue has been resolved by the team with commit [3283cef](#).

2. Mapping Declaration Lacks Named Parameters for Improved Readability

Severity: Informational

Category: Code Quality

Target:

- src/credits/ICDepositFactory.sol
- src/credits/ICInventoryVault.sol

Description

Since Solidity version 0.8.18, the language allows developers to add named key and value identifiers in mapping declarations using the syntax:

```
mapping(KeyType keyName => ValueType valueName)
```

This enhances code readability and clarifies how a mapping is intended to be used. In the ICInventoryVault contract, the mapping `orders` and `executionHashes` is declared without named parameters, making the purpose of the key and value less obvious upon inspection.

src/credits/ICInventoryVault.sol:L11-L20

```
contract ICInventoryVault is ICGoverned, EIP712, ReentrancyGuard {
    using SafeERC20 for IERC20;
    IERC20 public immutable token;
    uint256 public perTransferLimit;
    uint256 public perDayLimit;
    mapping(uint256 => uint256) public dailyUsed;
    mapping(address => bool) public signers;
    // 0 pending/unseen, 1 executed, 2 cancelled. A cancelled unseen order stays
    cancelled.
    mapping(bytes32 => uint8) public orders;
    mapping(bytes32 => bytes32) public executionHashes;
}
```

Recommendation

Consider updating the mapping declaration to explicitly name the key and value identifiers, enhancing readability and making the mapping's purpose clearer.

Status

This issue has been resolved by the team with commit [3283cef](#).

3. Missing events for functions that change critical state

Severity: Informational

Category: Logging

Target:

- src/credits/ICIssuanceController.sol

Description

Events allow capturing the changed parameters so that off-chain tools/interfaces can register such changes that allow users to evaluate them. Missing events do not promote transparency and if such changes immediately affect users' perception of fairness or trustworthiness, they could exit the protocol causing a reduction in protocol users.

In the `ICIssuanceController`, events are lacking in the privileged setter functions in the constructor.

src/credits/ICIssuanceController.sol:L35-L43

```
contract ICIssuanceController is ICGoverned, ReentrancyGuard {
    constructor(address multisig, IICMintable asset, address vault, uint256 cap)
        ICGoverned(multisig)
    {
        if (vault == address(0) || cap == 0) revert InvalidConfiguration();
        token = asset;
        inventoryVault = vault;
        supplyCap = cap;
    }
}
```

Recommendation

It is recommended to emit events for critical state changes.

Status

This issue has been resolved by the team with commit [3283cef](#).

Appendix

Appendix 1 - Files in Scope

This audit covered the following files in commit [b14bb19](#):

File	SHA-1 hash
Devfun.sol	f80734c799b5ab07ef6f5741f8f5fbb23f890b5c
ArenaWalletRegistry.sol	ed2e2e3f77c9f7b15b50bd9883c7a93571e0df30
ICBurner.sol	6cb68e9ac8d74b86f65323d20db58d7b022d0319
ICIssuanceController.sol	3b302cb007b56b603d7fd3b38c6e4d3ef5f91ce2
ICDepositFactory.sol	c6b8c20a62ec1dda913a7652d3131a0d16008d11
ICGoverned.sol	44a444a4fa5951658b6526abe3d0028ec0d15799
ICInventoryVault.sol	804108615da3450bf5ab6e273300aa5604737cd4